Kafka Usecases

Parameter	1. Race Condition Fix (Sync)	2. Parking Lot (Retry)	3. Domain Events (Audit)	4a. Critical Alerts (Wire/ Fraud)	4b. Marketing Alerts (Offers)
Topic Name Strategy	sync.customer.updates.v1	sync.customer.dlq.v1	domain.audit.logs.v1	notify.critical.v1	notify.marketing.v1
num.partitions	**Medium (e.g., 30)** *(Matches consumer thread count)*	**Low (e.g., 6)** *(Low concurrency needed)*	**High (e.g., 60+)** *(High write volume & concurrent readers)*	**Low/ Medium (e.g., 12)** *(Focus on low latency)*	**High (e.g., 50+)** *(Focus on massive parallel sending)*
cleanup.policy	delete	delete	delete	delete	delete
retention.ms	**3 Days** *(Ephemeral data)*	**14 Days** *(Allow time for manual fix)*	**90 Days** *(Audit Trail / Replay)*	**7 Days** *(Investigation window)*	**2 Days** *(Stale offers are useless)*
compression.type	lz4 (Fastest)	snappy (Balanced)	**zstd** (Best Ratio) *(Saves $$ on long retention)*	lz4 (Lowest Latency)	gzip or zstd (High throughput)
max.message.bytes	Standard (1MB)	Standard (1MB)	Standard (1MB)	Standard (1MB)	**Large (e.g., 5MB)** *(For rich HTML templates)*

segment.ms	7 Days	7 Days	**1 Day** *(Better for granular deletion)*	7 Days	4 Hours

1. Why zstd compression for Domain Events?
Infra Challenge: "Why not just use Snappy everywhere? It's standard." **Your Defense:** "Audit logs are highly repetitive JSON text stored for 90 days. zstd offers significantly higher compression ratios than Snappy, which will save us terabytes of disk storage cost over the retention period. The CPU trade-off is negligible for this async use case."
2. Why specific Partition Counts?
Infra Challenge: "Let's just give every topic 50 partitions." **Your Defense:** "For the **Parking Lot**, 50 partitions is wasteful because we will likely only have 1 single consumer replaying messages. It adds overhead to the brokers. Conversely, for **Marketing**, we need *more* partitions because we might spin up 100 consumer instances to blast out 1 million emails in 10 minutes. The partition count must match our max consumer parallelism."
3. Why retention.ms varies so much?
Infra Challenge: "Can we stick to a standard 7-day retention?" **Your Defense:** "For **Audit**, 7 days is insufficient if we need to rebuild a corrupted database table from last month. For **Marketing**, 7 days is a risk—we don't want to accidentally send a 'Flash Sale' email 5 days after the sale ended because the consumer was lagging. We want those messages to expire quickly."

Producer-Side Implications (The "Hidden" Specs)

While the above are *Topic* configs, the "Quality of Service" is actually dictated by how your application (the Producer) sends the data. You should add this small addendum for your developers:

- **For Critical Alerts & Sync:** Must set acks=all (Wait for all replicas to confirm). This guarantees zero data loss but is slightly slower.
- **For Marketing Alerts:** Set acks=1 (Wait for leader only). This is much faster; if we lose 1 offer email in a million during a server crash, it is acceptable.
- **For Domain Events:** Use linger.ms=20 (Batching). Wait up to 20ms to group events together before sending. This drastically improves network throughput for high-volume logs.

#	Use Case	Implementation Pattern	Payload & Volume Profile	Storage & Retention Needs	Capacity/ Infra Impact
1	**Race Condition Fix** *(Marketo/ Okta Sync)*	**Serialization** *(Keyed Partitioning)*	**Size:** Small (< 5KB) **Vol:** Low/ Medium (User actions) **Flow:** Steady Stream	**Retention:** Short (e.g., 3 Days) *Data only needed until sync completes.*	**Constraint: Strict Ordering** required. **Impact:** Requires carefully calculated partition counts. Too few = slow processing; Too many = idle threads.
2	**Resiliency / Retry** *(API Limit Parking Lot)*	**Dead Letter Queue (DLQ)** *(Delayed Consumption)*	**Size:** Medium (Payload + Headers) **Vol:** Zero normally; **High Burst** during outages. **Flow:** Spiky	**Retention:** Long (7+ Days) *Must hold data safely while external API is down (potentially 24h+).*	**Constraint: Disk I/O.** **Impact:** During an outage, this topic acts as a buffer. Needs sufficient disk space to absorb 24h of backlog without choking brokers.

3	**Domain Auditing** *(Bank Assist Logging)*	**Fire-and-Forget** *(Async Decoupling)*	**Size:** Small (JSON Events) **Vol: High** (Every click/action) **Flow:** Heavy Constant Write	**Retention:** Very Long (30-90 Days) *Needed for "Replay" capability if Audit DB crashes or needs rebuilding*.	**Constraint: Throughput & Fan-Out**. **Impact:** High network bandwidth usage. Likely multiple consumer groups (DB Sync, Splunk, Analytics) reading the same data simultaneously.

4	**Unified Alerts** *(Notifications)*	**Priority Queues** *(Fan-In Aggregation)*	**Size:** Large "Fat Events" (10-50KB) **Vol:** Mixed (Transactional = Steady; Marketing = **Massive Spikes**) **Flow:** Highly Variable	**Retention:** Medium (7 Days) *Enough time to investigate "undelivered" complaints*.	**Constraint: Latency vs. Throughput**. **Impact:** Needs "Topic Isolation" (Separate topics for Critical vs. Marketing) to prevent Marketing spikes from causing Critical alert lag.

1. Why we need "Keyed Partitioning" (Row 1):
"We aren't just using Kafka for speed; we are using it as a distributed mutex. We need enough partitions to handle our user concurrency, but we can't just auto-scale partitions later without re-hashing our keys. We need to size this correctly upfront."
2. Why we need High Disk Space for the Parking Lot (Row 2):
"This topic will sit empty 99% of the time, but when Marketo goes down, it becomes a dam holding back the entire day's volume. We need to provision disk space for the 'Worst Case Scenario' (24 hours of downtime), not the average case."
3. Why we need Long Retention for Domain Events (Row 3):
"This is our 'Source of Truth.' If the Audit Database corrupts, we will use this Kafka topic to rebuild the table. We need a retention policy that allows for a reasonable recovery window (e.g., 30 days)."
4. The "Noisy Neighbor" Protection (Row 4):
"We are requesting separate topics for 'Critical Alerts' vs 'Marketing Offers'. Even though the data shape is similar, the traffic profile is different. We cannot allow a 500k batch of marketing emails to induce latency on Wire Transfer SMS alerts."

#	Use Case	Pattern Name	Key Benefit
1	**Direct Deposit / Marketo**	Serialization (Partition Key)	Prevents Race Conditions/ Duplicates.
2	**Marketo API Limits**	Parking Lot (Dead Letter Queue)	Retries failures without blocking live traffic.
3	**Domain Events (Audit)**	Async Decoupling (Fire & Forget)	fast user experience; eventual consistency for reporting.
4	**Notifications**	Fan-In / Priority Queues	Removes DB polling; isolates critical alerts from marketing spam.